西南交通大学
Southwest Jiaotong University

面向对象程序设计

开课学院：计算机与人工智能学院

课程学分：2.0（32学时）

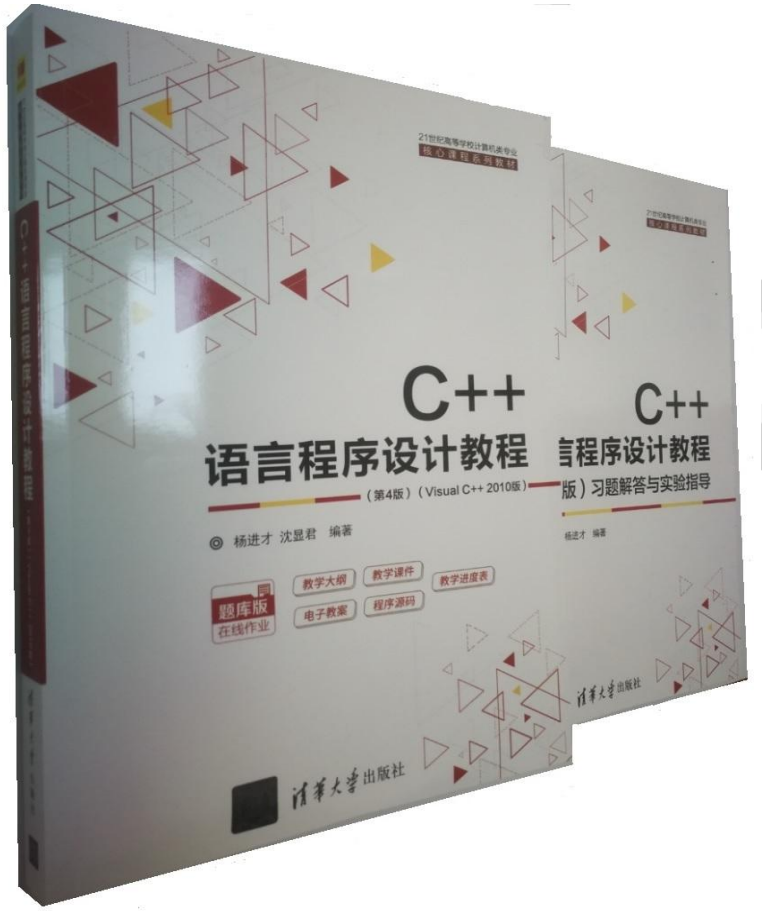
授课教师：廖革元

★前言—教材

C++语言程序设计教程（第4版）

作者：杨进才，沈显君

ISBN：9787302567561



课堂教学内容源于教材，但不限于教材，是相关内容的提炼，着重体现面向对象程序设计思想及其应用。

★课程目标—人人都会编程

1. 通过对面向对象的基本概念和机制的学习，能够对一般计算机软件系统进行建模，掌握以对象思维分析问题和解决问题的能力。（对应毕业要求指标点1.2）。
2. 掌握面向对象程序的一般结构，具备以对象方法对具体需求进行设计与实现的能力（对应毕业要求指标点3.1）。
3. 能够根据实际需求充分利用现有开发工具的优势和特点解决具体问题，具备对工具的选择和应用的能力（对应毕业要求指标点5.1）。

★课程主要教学内容及范围

1. **基础部分：** cin/cout、引用、函数重载、函数参数默认值、内联、new/delete、异常处理初步等。
2. **类和对象（封装机制）：** 类的定义方式，类成员使用，对象创建和使用，构造函数和析构函数，复制构造函数，私有构造和析构，static成员，友元，this指针。
3. **继承及组合（继承机制）：** 派生类（子类）定义及使用；继承种类（单、多、重复、虚）和继承方式（公有、私有、受保护）；父类（基类）、子类（派生类）等基础概念；父类成员在子类中的表现；子类（整体类）构造函数和析构函数；覆盖及同名成员问题；复杂情况下子类（整体类）构造函数和析构函数的执行顺序。
4. **多态及虚函数（多态机制）：** 虚函数，虚析构函数，虚函数和非虚函数的相互调用，纯虚函数与抽象类；运算符重载；函数模板与类模板基础。
5. **其他：** 程序运行过程中隐含的构造函数和析构函数调用（对象创建和撤销时，函数调用时的对象参数，函数返回对象，继承时）；面向对象程序的一般结构，优势，程序升级思想及步骤，QT等应用程序编程，组件思想，面向对象分析与设计方法初步等。

本课程配套使用的部分教学资源如下：

[1] 杨进才，沈显君编著. C++语言程序设计教程(第4版). 清华大学出版社，2022.1.

[2] 李晋江 编著. C++面向对象程序设计(第2版). 清华大学出版社，2016.1.

[3] 中国大学MOOC(慕课) 国家精品 C++程序设计 西北工业大学 魏英老师

https://www.icourse163.org/course/NWPU-494001?from=searchPage

[4] 中国大学MOOC(慕课) 面向对象程序设计 南开大学 王恺老师

https://www.icourse163.org/course/NKU-1206155803? from=searchPage l

[5]各教学班建立QQ学习群

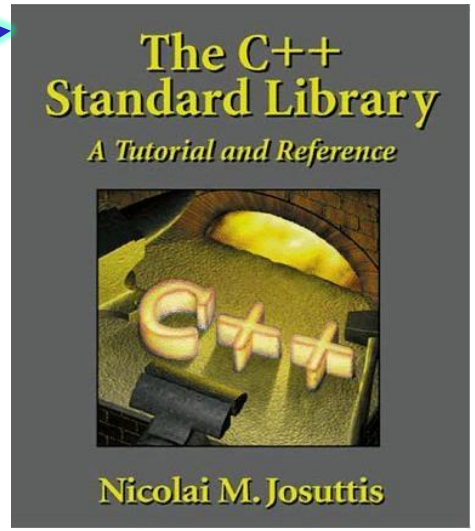
教学资源不限于以上教材、网络资源和方式。

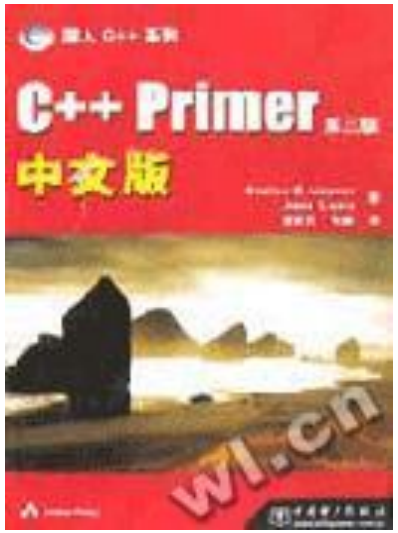
★面向对象程序设计课程考核及成绩组成

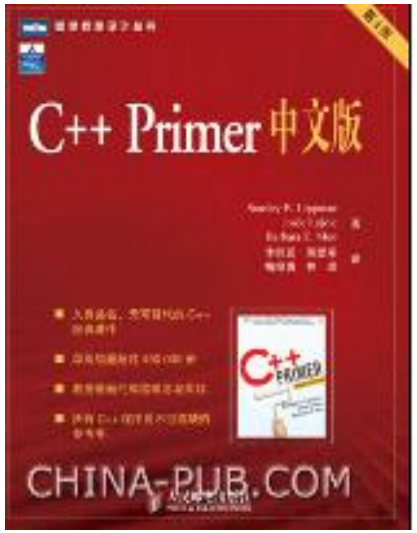
考核方式	考核详情	占总成绩比例
期末笔试	笔试试卷	50%
课堂表现	课堂考勤及课堂交互	10%
在线学习	通过头歌等线上实验平台学习，完成线上作业和实验	15%
课程设计	课程设计报告、源程序、答辩	15%
半期考试	期中考试	10%

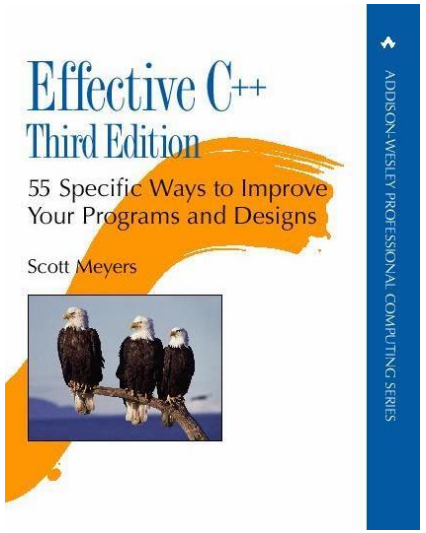
★前言—本课程配套使用的部分教学资源

- 课件**绝不是**学习C++的唯一资料，你需要几本好书
  - 《C++ Primer》1000页
  - 《Effective C++》3th 400页
  - 《C++标准程序库》 800页
  - 如果你自认为基础较差，还有一本**较浅的书**









知识靠书本给你，经验则要在**实践**中慢慢积累。**多阅读**，才能精通，否则只能孤陋寡闻。

★关于C++语言

- TIOBE编程社区指数是衡量编程语言流行度的指标。该指数每月更新一次，是基于谷歌、必应等25种搜索引擎上与编程相关的搜索查询得到的结果。TIOBE指数旨在反映编程语言受欢迎程度的变化。TIOBE公布了2025年2月编程语言排行榜，Python排名第一，占比 23.88%，C++排行第二，比11.37%。

Feb 2025	Feb 2024	Change	Programming Language	Ratings	Change
1	1		Python	23.88%	+8.72%
2	3	↑	C++	11.37%	+0.84%
3	4	↑	Java	10.66%	+1.79%
4	2	↓	C	9.84%	-1.14%
5	5		C#	4.12%	-3.41%
6	6		JavaScript	3.78%	+0.61%
7	7		SQL	2.87%	+1.04%
8	8		Go	2.26%	+0.53%
9	12	↑	Delphi/Object Pascal	2.18%	+0.78%
10	9	↓	Visual Basic	2.04%	+0.52%
11	11		Fortran	1.75%	+0.35%
12	15	↑	Scratch	1.54%	+0.36%
13	18	⇈	Rust	1.47%	+0.42%
14	10	⇊	PHP	1.14%	-0.37%

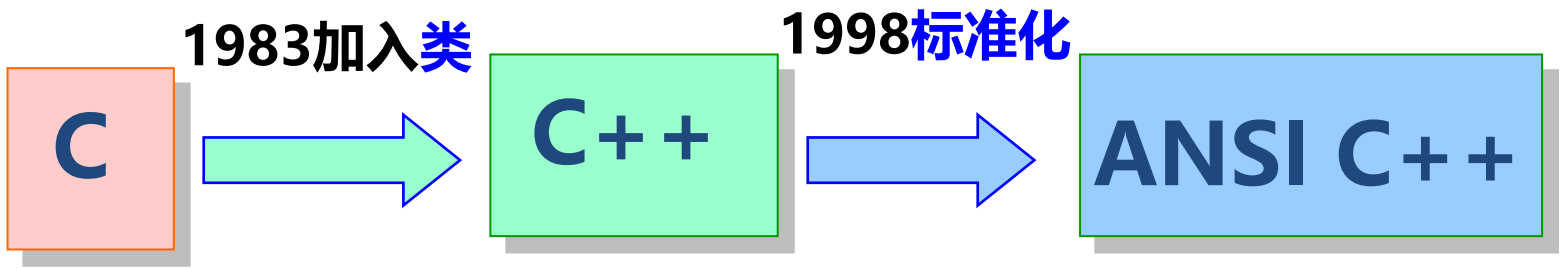
★关于C++语言

C++、C、Java、Python、C#语言的使用

语言	主要用途
C	操作系统、嵌入式、驱动开发
C++	图形图像、科研、通信、桌面软件、游戏、游戏服务器
C#	Windows桌面软件、.Net web、服务器
Java	Java SE：跨平台的桌面应用，Android
	Java EE：企业级应用、Web开发、服务器后端
	Java ME：手机应用，流利于非智能机时代
	Java Android：用于开发安卓应用
Python	Web、科学计算、运维

★关于C++语言

C++语言的产生



C++语言的特点

- 支持面向对象的程序设计（抽象、封装、继承、多态）
- 支持泛型程序设计 (类模板、函数模板)
- 功能强大的标准模板库 （STL）
- C++语言是一种更好的“C语言” （新特性）

常用的C++集成开发环境

- Visual C++
- Dev C++
- Borland C++Builder
- CodeBlock
- QT

★关于C++语言

C++与C、Java、Python 、C#语言的关系

- C++是从C语言发展演变扩充而来，是“带类的”更好的C。C是C++的一个子集，C语言程序不加修改可以在C++编译器中编译。因此，C++既可以C语言为基础，也可以单独学习。
- C++、Java、C#和Python都是面向对象的编程语言。C++更接近于底层，编译成本地机器码，由操作系统直接运行，运行效率高。
- Java在C++的基础上做了更高级的封装。Java编译成字节码，由Java虚拟机读取运行，一次编译到处运行，运行效率偏低。
- Python也是基于C++创造的一种脚本语言。简单易学，编程效率高，用解释方式运行。尽管Python在执行效率上不如其他一些语言，但其易学性使其成为许多非软件工程师的首选。
- C#在微软在C++基础上发展出来的，限制掉了C++的一些东西,又增加了一些如内存自动回收等功能，是限制了的C++版。是为了提升开发效率和软件工业化的产物。

★再谈程序设计方法

1. 结构化程序设计的思考

□ 结构化程序设计

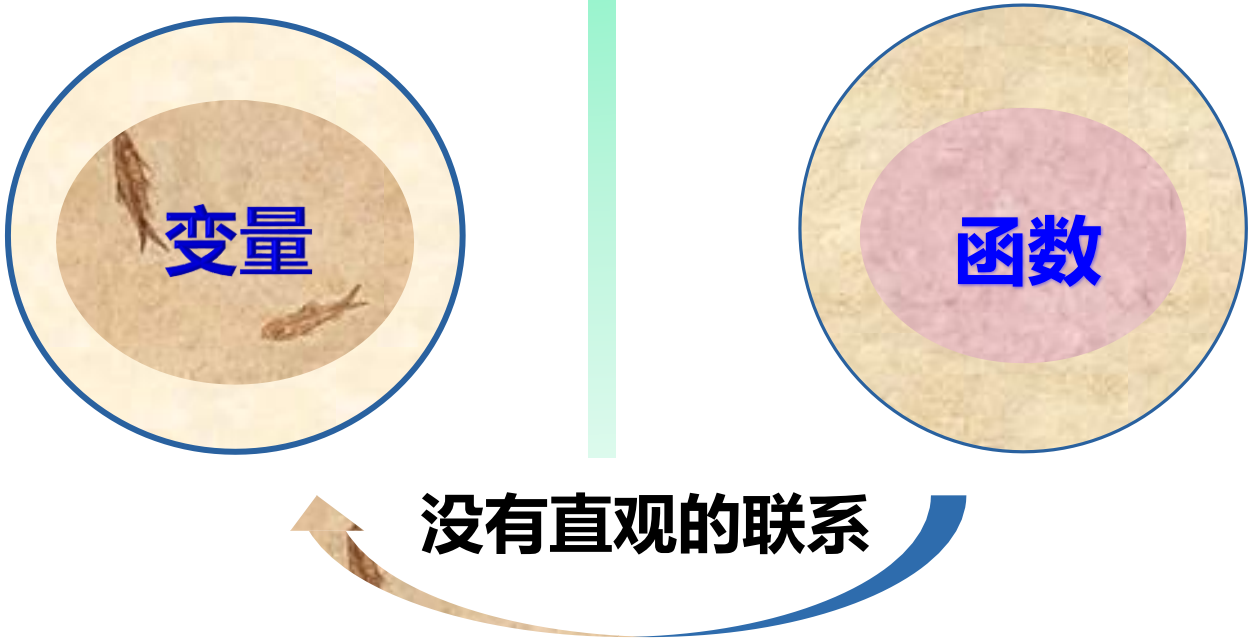
复杂的大问题 ⇨ 层层分解/模块化 ⇨ 若干子问题

结构化程序设计采用自顶向下、逐步细化、功能模块化的方法。



2. 结构化程序设计的问题

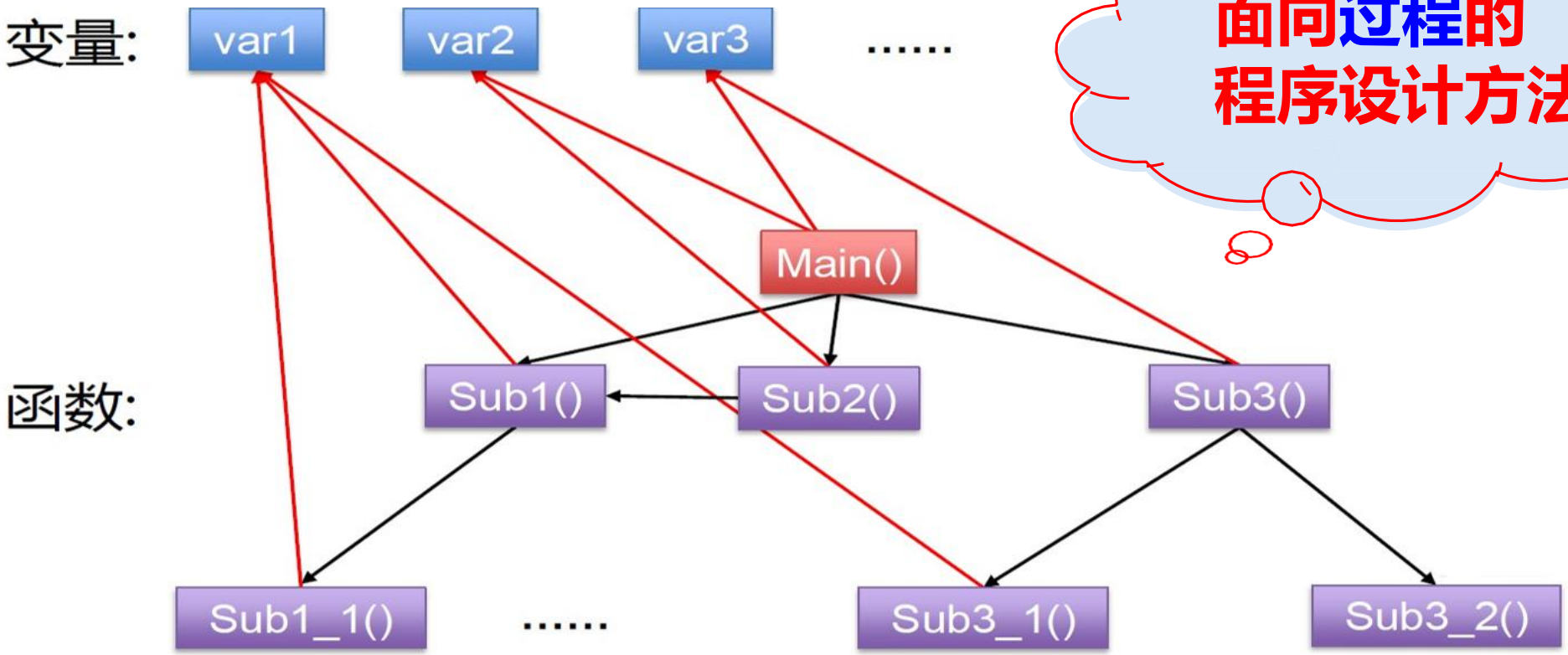
- 在结构化程序设计中





★再谈程序设计方法

3. 结构化程序设计的程序模式



面向过程的
程序设计方法

困境

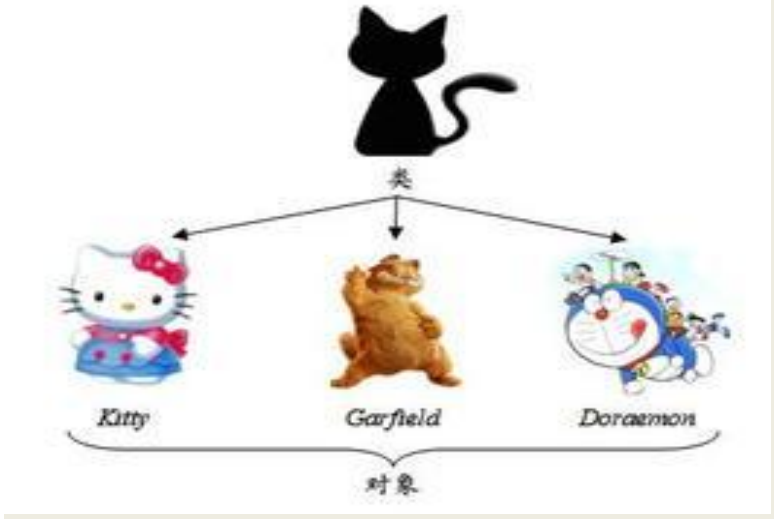
理解难 → 修改难 → 差错难 → 重用难

再谈程序设计方法

1、面向对象的程序设计思想

类(class):

对象(object):



面向对象的程序设计=类+类+…+类

设计**程序**的过程 ⇒ 设计**类**的过程

一类事物 ⇒ 共同属性 ⇒ 数据；行为 ⇒ 函数

抽象

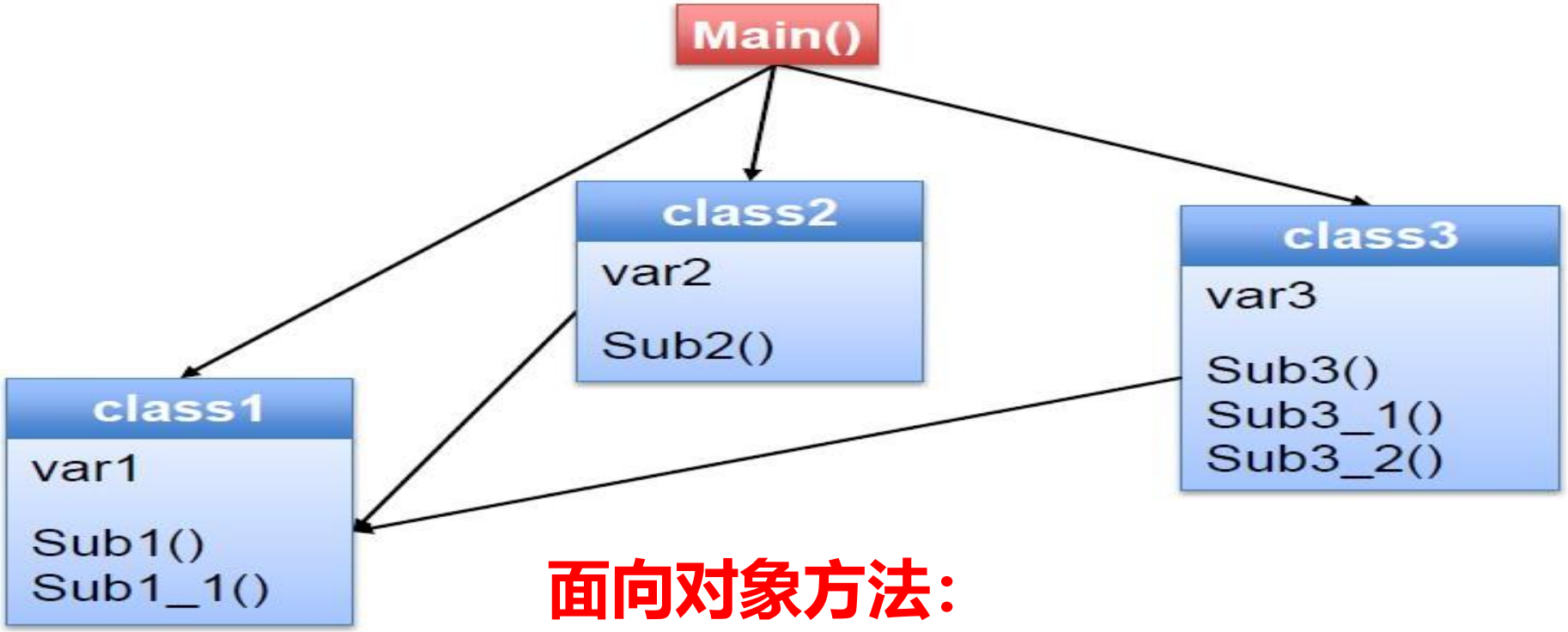
封装

类

```
class
Data
function
```

再谈程序设计方法

2、面向对象程序设计方法



面向对象方法：

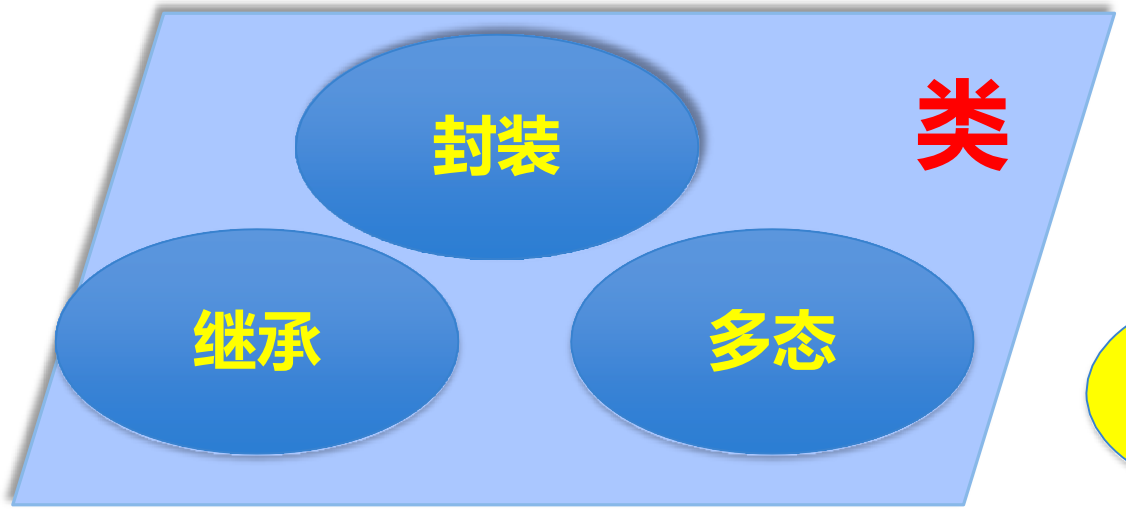
- 以数据（对象）为中心组织程序模块
- 方法体（函数体）仍然是面向过程的
- 对象间通过消息传递实现相互通信

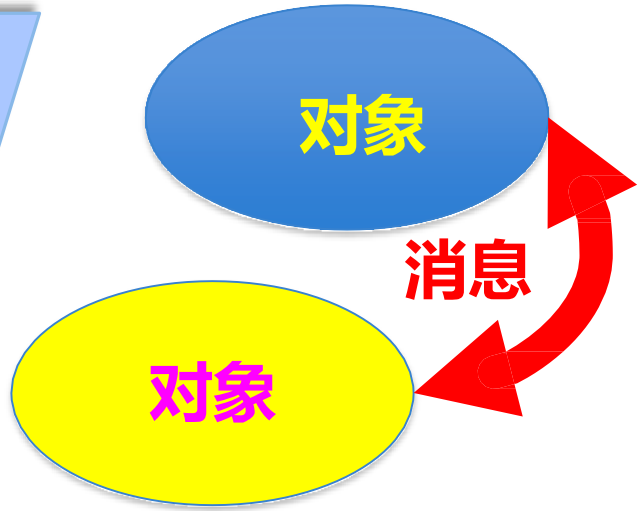
面向对象程序设计方法（侧重解决业务问题）

我们需要掌握面向对象编程的思想→如何用类的思想描述事物

OOP（Object-Oriented Programming，面向对象编程）：

- 将数据及对数据的操作方法放在一起，作为一个相互依存、不可分离的整体（称为对象）。
- 对同类型对象抽象出其共性，形成类：类中的大多数数据，只能用本类的方法（函数）进行处理（封装性）。
- 类通过一个简单的外部接口与外界发生关系，对象与对象之间通过消息进行通信。

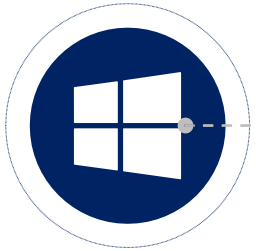






★程序设计方法的延伸

COP



面向组件编程

组件技术更核心的概念是接口。主要目标是复用 - - 粗粒度的复用，如一个dll(动态链接库)、一个中间件，甚至一个框架。

AOP



面向方面编程

实现分散关注的编程方法，它将“关注”封装在“方面”中……，例如日志管理、权限管理

SOP



面向服务编程

一个服务是服务消费者获得其想要的最终结果的一个工作单元。服务者与消费者都以软件代理代表他们自己的角色。

如天气预报 、火车时刻查询……

【面向过程程序设计】

比如，有下面的需求：

张三的语文书的第38页的第1个字符是什么?
李四的数学书的第50页的第8个字符是什么?
王五的外语书的第23页的第20个字符是什么?

需求：

- **谁的书**
- **什么书**
- **第多少页**
- **第多少个字符**

结构化抽象的例子

```
char getx(char *who,char *book,int page,int pos)
{    //函数具体操作      }
void main()
{
        getx("张三","语文",38,1);
        getx("李四","数学",50,8);
        getx("王五","外语",23,20);
}
```

【面向对象程序设计】

面向对象程序设计方法主张以对象为单位。需求中的人（张三、李四等）都是对象，不同的书也都是对象，每个对象彼此独立存在，互不干扰。

```
面向对象抽象的例子
void main()
{
        cout << 张三.语文.C[38][1];
        cout << 李四.数学.C[50][8];
        cout << 王五.外语.C[23][20];
}
```

在该例子中，张三的语文和李四的语文完全可以不同，彼此独立，能够很好地实现个体维护而不影响其他。

【面向对象的不足】

(1) 程序的编译和执行对存贮空间的要求较高。

(2) OOP程序的效率在很多时候不如同样规模的面象过程开发的程序。

(3) 针对具体应用建立一套类体系的工作量大

(4) OOP的很多概念和编程方式初学者不易理解和入门。



面向对象程序设计的一般过程

面向对象的思想是以类为主导的。其过程如下：

一、构造类

二、构建类和类之间的关系

三、创建应用类对象

四、使用对象



例：输出一行字符: This is a C++ program.

```
#include<iostream>
using namespace std;
 int main()
{
    cout<<"This is a C++ Program. " << endl;
    return 0;
}
```

实例：用C++写一个1到100累加的程序

面向对象程序设计

C++新特性

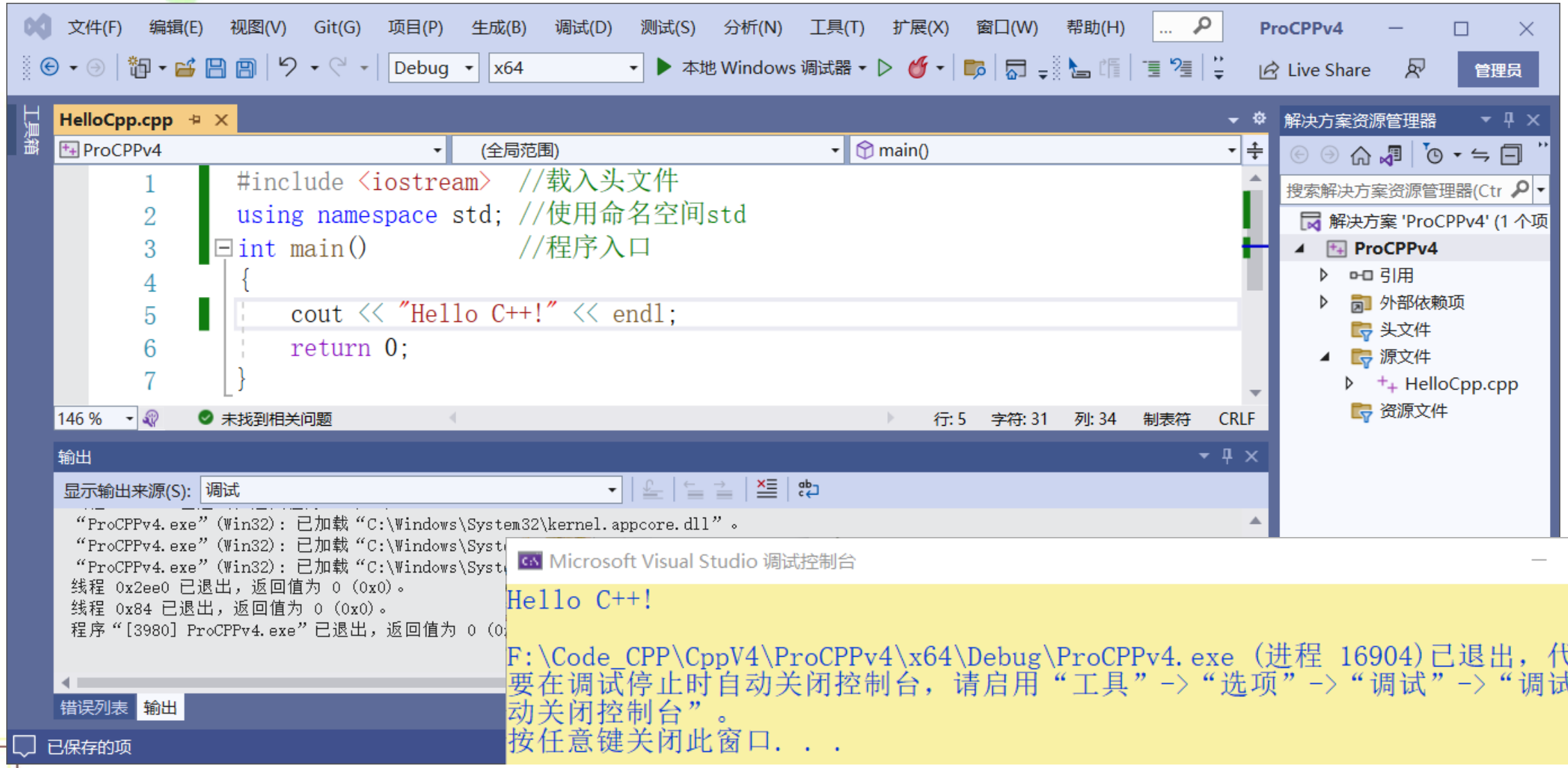
★第1-4章 C++与C基础知识点差异提炼

西南交通大学
Southwest Jiaotong University

C++的新特性

1. C++数据的输入输出 (1.4.3节)
2. 名字空间namespace (1.4.2节)
3. 引入新的数据类型(2.1.2节、10章、11章)
4. 带默认参数的函数 (2.5.3节)
5. 内联函数 (2.5.4节)
6. 函数重载 (2.5.5节)
7. 内存管理 (3.3.5节)
8. const (3.3.7节)
9. C++引用 (3.4节)
10. 异常处理 (12章)

一个简单标准的C++程序（Hello C++）（教材P.11 例1-1）

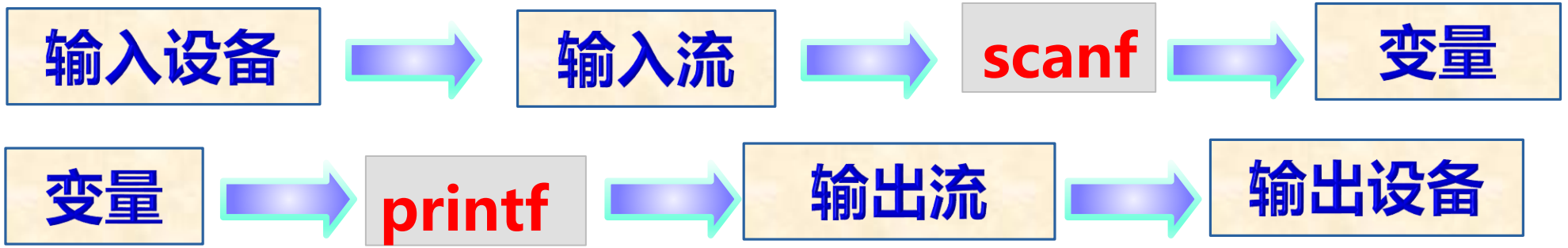


★C++的关键字表

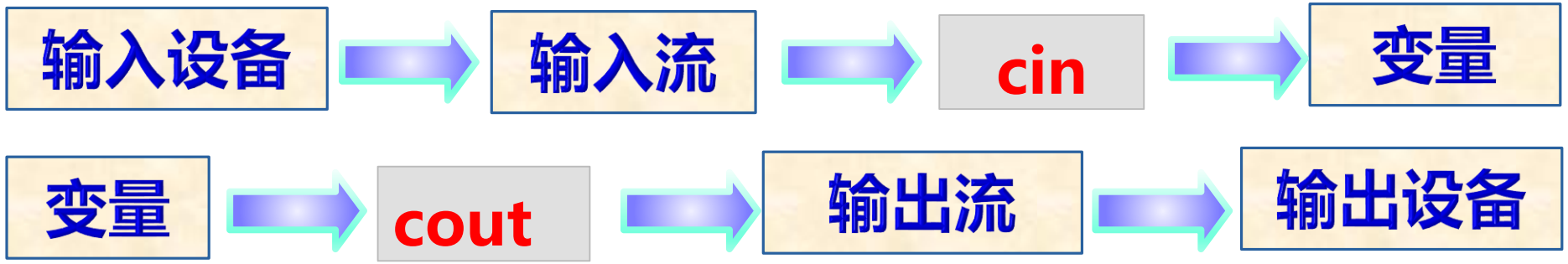
asm	const_cast	explicit	inline	public	struct	typename
auto	continue	export	int	register	switch	union
bool	default	extern	long	reinterpret_cast	template	unsigned
break	delete	false	mutable	return	this	using
case	do	float	namespace	short	throw	virtual
catch	double	for	new	signed	true	void
char	dynamic_cast	friend	operator	sizeof	try	volatile
class	else	goto	private	static	typedef	wchar_t
const	enum	if	protected	static_cast	typeid	while

数据的输入与输出形式发生变化

- C 语言的输入/输出方式



- C++的输入/输出方式



★C++新特性： 数据的输入输出

西南交通大学 Southwest Jiaotong University

cout输出

语法格式： cout{<<表达式}*

其中，{ … }*表示{}中内容可以重复1~若干次

<<称为插入运算符

如：float a=3.45f; int b=5; char c='A';

```
cout<<"a="<<a<<","<<"b="<<b<<","<<"c="<<c;
```

输出结果是：a=3.45,b=5,c=A



两个预定义控制符：

cout<<endl; cout<<"\n";

cout<<ends; cout<<"\0";

注意：插入ends显示效果同插入空格cout<<"\x20"，但ends不是空格。

插入列宽控制： setw(n) //#include <iomanip>

指定其后插入的数据项的字符列宽，若数据项长度小于列宽，数据左边补足空格，否则按实际位数输出。

例：

```
cout<<"a="<<setw(6)<<15;
```

输出结果为：a= 15

★C++新特性： 数据的输入输出

西南交通大学 Southwest Jiaotong University

□ cin输入

语法格式： cin{>>变量名}*

其中，{ … }*表示{}中内容可以重复1~若干次

>>称为提取运算符



例：float a; int b; cin>>a>>b;

cin输入字符(提取char型变量)时，空格/TAB/回车换行是分隔符;

例：char a,b,c,d;

cin>>a>>b>>c>>d; cout<<a<<b<<c<<d;

输入为：A空格BTABC回车D回车

输出为：ABCD

★C++新特性： 数据的输入输出

西南交通大学 Southwest Jiaotong University

cin输入字符串(提取到char *型变量或string变量)时，等效于scanf的%s格式。

3. 使用cout和cin的优缺点

优点：无需指定格式， 由变量数据类型自动转换

缺点： 效率低于scanf和printf； 输入/输出格式控制比较麻烦。

4. cout和cin应用举例

例 输入打印行数n，打印如下图形。

```
    *
   ***
  *****
 *******
*********
```

```
#include <iostream>
#include <iomanip>
using namespace std;
void main(){
   int i,j,n;
  cout<<"Input n="; cin>>n;
  for(i=1;i<=n;i++){
     cout<<setw(n-i+1);
  for(j=1;j<=2*i-1;j++)  cout<<"*";
  cout<<endl;
 }}
```



★C++新特性： 数据的输入输出

西南交通大学 Southwest Jiaotong University

例：简单C++与C程序对比

C++程序

```
#include <iostream>//包含C++标准输入输出库
using namespace std;//使用命名空间std
int main() {
cout << "Hello World!"<<endl;//C++输出
return 0;
}
```

C程序

```
#include <stdio.h>

int main() {
printf("Hello World!\n");//C输出
return 0;
}
```

```
#include <iostream>
int main() {
    std::cout << "Hello World!" << std::endl;
    return 0;
}
```

如果不使用：using namespace std;
在程序中用到C++标准库时，需要使用std作为限定。

::域作用符 与名字空间结合使用

★C++新特性： 数据的输入输出

西南交通大学 Southwest Jiaotong University

例：输入整数a、b，输出a+b。

C++程序

```
#include <iostream>//包含C++标准输入输出库
using namespace std;//使用命名空间std
int main() {
    int a,b;
    cin>>a>>b;//输入两个整数a和b
    cout<<a<<"+"<<b<<"=";//输出a+b算式
    cout<<a+b<<endl;//输出a+b结果
    return 0;
}
```

C程序

```
#include <stdio.h>

int main() {
    int a,b;
    scanf("%d%d",&a,&b);
    printf("%d+%d=",a,b);
    printf("%d\n",a+b);
    return 0;
}
```

C++新特性：1.数据的输入输出

1.2 C++输出数据格式控制

(1) 以成员函数方式控制输出精度

```
cout.setf(ios::fixed);          //设置定点小数格式
cout.precision(2);              //两位小数
cout << 3.14159 << endl;
cout << 103.14159 << endl;
```

```
3.14
103.14
```

如果删除第一行，即不设置定点小数格式，输出结果是什么？
此时，cout.precision(2); 设置的整个数的输出精度为2。

```
3.1
1e+002
```

(2) 以操纵符(控制符)方式控制输出精度

```
cout.flags(ios::right);//以下格式均设置为右对齐
cout<<setw(10)<<setprecision(5)<<3.14159<<endl;
cout<<setprecision(3)<<3.14343<<endl;
```

```
   3.1416
3.14
```

C++新特性：1.数据的输入输出

常用的操纵符（P.15 表1-1）

所有字母小写

操纵符	作用	说明
oct	整数数据以8进制形式输出	作用范围为后续输出的整数对象，对小数不起作用
dec	整数数据以10进制形式输出(默认)	
hex	整数数据以16进制形式输出	
endl	换行并刷新输出流	
setw(n)	设置输出宽度	需包括头文件iomanip，仅对后一个对象起作用
setprecision(n)	设置输出小数位数(默认为6)	需包括头文件iomanip，作用范围为后续对象

☞注意:

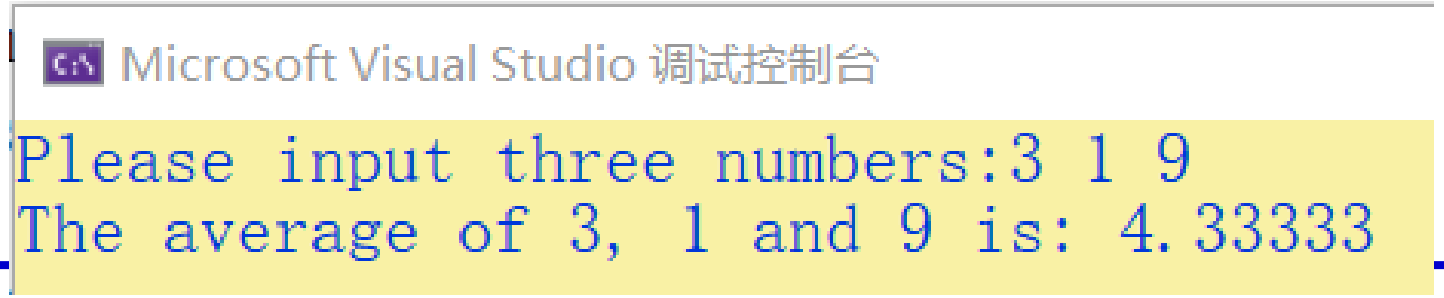
- 在使用setw(n)与setprecision(n)时，还需要包含输入/输出操纵符头文件iomanip，iomanip意为input and output manipulate（操纵）。
- setprecision(n)仅对带有小数的数有效，n为整数与小数但不包括小数点的位数。

C++新特性：1.数据的输入输出（例1-2）

```
//功  能：求3个数的平均值，演示C++的简单I/O
#include <iostream>//包含C++标准输入输出库
using namespace std;//使用名字空间std
int main() {
    float num1, num2, num3;  //定义3个数
    cout << "Please input three numbers:" ;
    cin >> num1 >> num2>>num3;//输入3个数
    cout << "The average of "<<num1<<", "<<num2<<" and "<<num3;
    cout <<" is: "<<(num1+num2+num3)/3<< endl;//输出结果
    return 0;
}
```

输入：每个变量前都有一个提取运算符>>

输出：字符串原样输出，表达式输出值



★C++新特性：命名空间(namespace)(P12)

命名空间（namespace）

```
using namespace std;
//标准命名空间，所有标识符都定义在std里面
```

为什么需要命名空间（避免名称冲突）



程序中的命名空间



想一想：C 软件公司如果购买了A公司和B公司的程序库，想使用A公司的变量x，B公司函数f1怎么办？

★C++新特性：命名空间(namespace)

西南交通大学 Southwest Jiaotong University

namespace（命名空间）—— 标准命名空间 std

- 命名空间是ANSI C++引入的可以由用户命名的作用域，用来处理程序中常见的同名冲突。
- C++引入了类作用域（命名空间），类是出现在文件内的。在不同的作用域中可以定义相同名字的变量，互不干扰，系统能够区别它们。
- 标准C++库的所有的标识符都是在一个名为std的命名空间中定义的，或者说标准头文件(如iostream)中函数、类、对象和类模板是在命名空间std中定义的。

★C++新特性：命名空间(namespace)

西南交通大学 Southwest Jiaotong University

在std中定义和声明的所有标识符在本文件中都可以作为全局量来使用。但是应当绝对保证在程序中**不出现与命名空间std的成员同名的标识符**。由于在命名空间std中定义的实体实在太多，有时程序设计人员也弄不清哪些标识符已在命名空间std中定义过，为减少出错机会，有的专业人员喜欢用多个“**using 命名空间成员**”声明来代替“**using namespace 命名空间**”声明。

```
using std::endl;     //using声明
namespace B
{
    int cout=2;
    int y=2;
}
int main(int argc, char *argv[])
{
    std::cout << "Hello World!" << endl;
    std::cout<<"hello"<<endl;
    using B::cout;
    std::cout<<cout<<endl;
}
```

:: 域作用符：与命名空间结合使用